

EVALUACION	Obligatorio	GRUPO	TODOS	FECHA	15/09/25 V1
MATERIA	Algoritmos y Estructuras de Datos				
CARRERA	Analista Programador / Analista en Tecnologías de la información				
CONDICIONES	Obligatorio 1 - Puntaje máximo: 20 puntos - Puntaje mínimo: 0 puntos - Fecha de entrega: 09/10/2025 hasta las 21:00 horas Obligatorio 2 - Puntaje máximo: 35 puntos - Puntaje mínimo: 0 puntos - Fecha de entrega: 19/11/2025 hasta las 21:00 horas Entregas se realizan en gestion.ort.edu.uy (max. 40Mb en formato zip, rar) Uso de material de apoyo y/o consulta <u>Inteligencia Artificial Generativa</u> - Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. - Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. - Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla. - Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de "alucinaciones", los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso - En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema IMPORTANTE: - Previo a la semana de cada entrega, quedarán disponible un juego de pruebas para validar todas las operaciones solicitadas. Adicionalmente, el grupo debe entregar su conjunto de pruebas (utilizando Junit4 y la clase Retorno provista) que evidencien el testing realizado. - Inscribirse. - Formar grupos de hasta 2 personas del mismo dictado. - Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: "RECORDATORIO") Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails alamon@ort.edu.uy y rodriguez_mb@ort.edu.uy o telefónicamente al 29021505 - int 1156 u 1138				

IMPORTANTE: Verificar que se haya recibido el correo electrónico corroborando la correcta entrega en gestión, y verificar que haya quedado entregada la última versión del obligatorio de forma correcta.

No se considerarán entregas fuera de fecha.

Información importante

- La **selección adecuada de las estructuras** para modelar el problema y la **eficiencia** en cada una de las operaciones es muy importante.
- Se deben implementar todos los TADs utilizados, no se permite el uso de estructuras nativas de JAVA (colecciones, pilas, colas, etc., por ejemplo, List, ArrayList, HashMap).
- Los obligatorios se realizan por equipos de hasta 2 estudiantes.
- Se **deberán respetar los tipos de retorno dados**.
- El sistema no debe requerir ningún tipo de interacción con el usuario o mensajes por consola.
- Es obligación del estudiante mantenerse al tanto de las aclaraciones que se realicen en clase o a través del foro de aulas.
- Deberá **aplicar la metodología vista en el curso**.
- El proyecto será implementado en lenguaje JAVA sobre una interfaz que se publicará en el sitio de la materia en aulas.ort.edu.uy (el uso de esta interfaz y del proyecto provisto es obligatorio).
- Defensa: la defensa del obligatorio será en fecha y formato a confirmar por el docente. La no asistencia a la defensa implica la pérdida de todos los puntos.

Obligatorio:

Contenido

1. Introducción - Sistema de Bicicletas Públicas (SiBiP)	4
2. Administración de Estudiantes y Libros	5
2.1. Crear Sistema de Gestión	5
2.2. Registrar Estación	5
2.3. Registrar Usuario	5
2.4. Registrar Bicicleta	5
2.5. Poner bicicleta en mantenimiento	6
2.6. Reparar bicicleta	6
2.7. Eliminar Estación	6
2.8. Asignar bicicleta a estación	7
2.9. Alquilar bicicleta	7
2.10. Devolver bicicleta	7
2.11. Deshacer últimos retiros	8
3. Listados y reportes	8
3.1. Obtener Usuario	8
3.2. Listar usuarios	8
3.3. Listar bicis en depósito	9
3.4. Mapa de estaciones	9
3.5. Listar bicis de estación	10
3.6. Estaciones con disponibilidad mayor	10
3.7. Ocupación promedio por barrio	10
3.8. Ranking por tipo de uso	11
3.9. Usuarios en espera por alquiler	11
3.10. Usuario con mayor cantidad de alquileres	11

1. Introducción - Sistema de Bicicletas Públicas (SiBiP)

El **Sistema de Bicicletas Públicas (SiBiP)** debe permitir gestionar las estaciones, bicicletas y alquileres/devoluciones que realizan los usuarios en diferentes áreas de Montevideo.

Cada estación del sistema cuenta con un conjunto de anclajes, que son los lugares disponibles para estacionar bicicletas. El número de anclajes determina cuántas bicicletas pueden estar simultáneamente en la estación. Por ejemplo, si una estación tiene 5 anclajes, podrá albergar como máximo 5 bicicletas.

Es importante tener en cuenta que, una vez ocupados todos los anclajes, **no se podrán asignar ni devolver más bicicletas en esa estación** hasta que se libere un espacio (es decir, hasta que un usuario retire alguna bicicleta). Si se intenta realizar una operación que supere la capacidad máxima, el sistema debe rechazarla devolviendo el error correspondiente

Además, el sistema brinda diferentes reportes de análisis para el seguimiento y estadísticas.

Se proveen los siguientes tipos de datos que deberán ser respetados:

Sistema	<pre>public class Sistema{ /*Aquí introduzca la información que estime conveniente*/ }</pre>
Retorno	<pre>public class Retorno{ public enum Resultado{OK,ERROR_1,ERROR_2,ERROR_3,ERROR_4, ERROR_5, ERROR_6, NO_IMPLEMENTADA}; boolean valorBooleano int valorEntero; String valorString; Resultado resultado; }</pre>

Pueden definirse tipos de datos (clases) auxiliares.

2. Administración de Estudiantes y Libros

2.1. Crear Sistema de Gestión

Firma: Retorno `crearSistemaDeGestion()`;

Descripción: Crea el sistema de gestión inicializando las estructuras a utilizar.

Retornos posibles	
OK	Si pudo inicializar el sistema correctamente.
ERROR	
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.2. Registrar Estación

Firma: Retorno `registrarEstacion(String nombre, String barrio, int capacidad)`;

Descripción: Crea una estación con nombre único, barrio y capacidad de bicicletas > 0. Inicialmente, **0** bicis en anclajes.

Retornos posibles	
OK	Si pudo registrar la estación en el sistema.
ERROR	1: Si alguno de los parámetros es null o vacío. 2: capacidad <= 0 3: estación ya existe
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.3. Registrar Usuario

Firma: Retorno `registrarUsuario(String cedula, String nombre)`;

Descripción: Registra el usuario en el sistema. Cédula: exactamente de 8 dígitos. El nombre no puede estar vacío.

Retornos posibles	
OK	Si pudo registrar el usuario en el sistema.
ERROR	1: Si alguno de los parámetros es null o vacío. 2: formato cédula inválido 3: ya existe usuario con esa cédula
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.4. Registrar Bicicleta

Firma: Retorno `registrarBicicleta(String codigo, String tipo)`;

Descripción: Se registra una nueva bicicleta de código único (6 caracteres) y tipo: URBANA, MOUNTAIN, ELECTRICA}. La bicicleta se registra en depósito inicialmente (no asignada a estación).

Retornos posibles	
OK	Si pudo registrar la bicicleta
ERROR	1: Si alguno de los parámetros es null o vacío. 2: Formato incorrecto de código 3: El tipo no está dentro de los permitidos 4: Ya existe bici con ese código
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.5. Poner bicicleta en mantenimiento

Firma: Retorno `marcarEnMantenimiento(String codigo, String motivo);`

Descripción: Solo si la bici no está alquilada. Pasa a estado "Mantenimiento" y queda fuera de servicio, siendo retirada y dejada en el depósito (en caso de no estarlo actualmente).

Retornos posibles	
OK	Si se pudo poner la bicicleta en mantenimiento
ERROR	1: Si alguno de los parámetros es null o vacío. 2: bici inexistente 3: bici actualmente alquilada 4: bici ya en mantenimiento
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.6. Reparar bicicleta

Firma: Retorno `repararBicicleta(String codigo);`

Descripción: Repara la bici en mantenimiento dejándola en el depósito como "Disponible".

Retornos posibles	
OK	Si se pudo prestar el libro.
ERROR	1: Si alguno de los parámetros es null o vacío. 2: bici inexistente 3: bici no se encuentra en mantenimiento
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.7. Eliminar Estación

Firma: Retorno `eliminarEstacion(String nombre);`

Descripción: Solo si **no tiene** bicicletas ancladas, ni reservas/colas pendientes.

Retornos posibles	
OK	Si se pudo eliminar estación
ERROR	1: Si alguno de los parámetros es null o vacío. 2: no existe estación 3: tiene bicis ancladas/colas pendientes
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.8. Asignar bicicleta a estación

Firma: Retorno `asignarBicicletaAEstacion(String codigo, String nombreEstacion);`

Descripción: Mueve la bici desde el depósito (o desde otra estación) a nombreEstacion si hay **anclaje libre** y la bici está **disponible**.

Retornos posibles	
OK	Si pudo asignar la bicicleta a la estación
ERROR	1: Si alguno de los parámetros es null o vacío. 2: bici no existe o no está "disponible" 3: estación no existe 4: estación sin anclajes libres
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.9. Alquilar bicicleta

Firma: Retorno `alquilarBicicleta(String cedula, String nombreEstacion);`

Descripción: Si hay bici **disponible** en la estación, asignarla y marcar "Alquilada". Si no hay, el usuario queda en espera en la estación hasta que llegue una. Se priorizará el orden para asignar las bicicletas, entre aquellos que están esperando.

Retornos posibles	
OK	Si pudo alquilar la bicicleta
ERROR	1: Si alguno de los parámetros es null o vacío. 2: usuario inexistente 3: estación inexistente
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.10. Devolver bicicleta

Firma: Retorno `devolverBicicleta(String cedula, String nombreEstacionDestino);`

Descripción: Devuelve bici del usuario en la estación destino. Si **no hay anclaje libre**, el usuario **queda en espera por un anclaje** (hasta que se libere). Al anclar: marcar bici "disponible" y, si hay usuarios en **espera de alquiler** en esa estación, **entregar automáticamente** al primero que estaba esperando (no aumenta stock visible, pasa directo a "alquilada").

Retornos posibles	
OK	Si pudo alquilar la bicicleta
ERROR	1: Si alguno de los parámetros es null o vacío. 2: usuario inexistente o no tiene bici alquilada 3: estación destino inexistente
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

2.11. Deshacer últimos retiros

Firma: Retorno deshacerUltimosRetiros(int n);

Descripción: Deshacer las últimas n asignaciones de alquiler realizadas en el sistema. Efecto inverso: devuelve la bici a su estación de origen (si hay lugar; si no, queda en cola de anclaje de esa estación), y revierte el alquiler. Cargar en valorString los n retiros deshechos en orden de deshacer.

Retornos posibles	
OK	Si pudo deshacer los retiros
ERROR	1: n <= 0
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

Formato: codigoBici#cedula#estacionOrigen|....

3. Listados y reportes

Los listados y reportes deben cargarse en el valor string del resultado Retorno, separando cada dato por un “#” y cada conjunto de datos por un “|” (Ver ejemplos).

3.1. Obtener Usuario

Firma: Retorno obtenerUsuario(String cedula);

Descripción: Devuelve el usuario de la cédula indicada

Retornos posibles	
OK	Si se retorna el listado correctamente.
ERROR	1: Si alguno de los parámetros es null o vacío. 2: formato cédula inválido 3: usuario inexistente
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

EJEMPLO DE CARGA DE RESULTADO:

Formato: nombre#cedula

Ejemplo: “Martina#12343337”

3.2. Listar usuarios

Firma: Retorno listarUsuarios();

Descripción: Se listan todos los usuarios ingresados en el sistema, ordenados creciente por nombre

Retornos posibles	
OK	Si se muestran todos los usuarios registrados ordenados
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno No por defecto.

Formato: nombre#ci|nombre#ci

Ejemplo: Martina Gutierrez#12344445|Pablo Lopez#43332221|Romina Mendez#13345556

3.3. Listar bicis en depósito

Firma: Retorno `listarBicisEnDeposito()`;

Descripción: Se listan las bicicletas que se encuentran actualmente en el depósito, en el orden en el cual fueron ingresadas (de muestra primero la que fue ingresada primero en depósito), con el detalle de si están en mantenimiento o no.

Retornos posibles	
OK	Si se muestran todas las bicicletas del depósito
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno No por defecto.

NOTA: debe ser implementada en forma recursiva.

Formato: `codigo#tipo#estado|codigo#tipo#estado`

Ejemplo: `AER345#URBANA#Disponible|UTR112#ELECTRICA#Mantenimiento`

3.4. Mapa de estaciones

Firma: Retorno `informaciónMapa(String [][] mapa)`;

Descripción: A partir de un mapa en donde se encuentran ubicadas diferentes estaciones.

Retornos posibles	
OK	Si retorna la información cargando los datos de las dos consultas en el valorString del retorno separadas por un
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

- 1) Indicar el máximo de estaciones en una misma fila o columna dentro del mapa. Adicionalmente al retorno agregar (separando por un #) si el máximo lo determinó una fila, columna o ambas. En caso de estar vacía indicar 0#ambas.
- 2) Si existen en algún caso, 3 columnas consecutivas (de izquierda a derecha), cuya cantidad de estaciones es estrictamente ascendente (para el ejemplo dado, las columnas 1,2 y 3 cumplen y también las 2,3 y 4 cumplen)

Ejemplo 1

o	o	o	o	o	o
o	o	o	E3	o	o
o	o	o	o	o	o
E1	o	o	o	E5	o
o	o	o	o	o	o
o	o	E2	o	E6	o
o	o	o	o	E7	o
o	o	o	E4	o	o

3#columna|existe

Ejemplo 2

o	o	o	o	o	o
o	o	o	E3	o	o
o	o	o	o	o	o
E1	o	o	o	E5	o
o	o	o	o	o	o
o	o	E2	o	E6	o
o	o	o	o	o	o
o	o	o	E4	o	o

2#ambas|existe

Ejemplo 3

o	o	o	o	o	o
o	o	o	E3	o	o
o	o	o	o	o	o
E1	o	o	o	E5	o
o	o	o	o	o	o
o	o	E2	o	E6	o
o	E7	o	o	o	o
o	o	o	E4	o	o

2#ambas|no existe

3.5. Listar bicis de estación

Firma: Retorno `listarBicicletasDeEstacion(String nombreEstacion);`

Descripción: Se listan las bicicletas de la estación indicada, en orden creciente por código.

Retornos posibles	
OK	Si se pudo listar las bicicletas de la estación
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno No por defecto.

NOTA: Esta operación debe realizarse recorriendo una sola vez la estructura escogida.

Formato: `codigo|codigo|codigo`

Ejemplo: `AER345|UTR112|UYT123`

3.6. Estaciones con disponibilidad mayor

Firma: Retorno `estacionesConDisponibilidad(int n);`

Descripción: Indicar (cantidad) de estaciones que cuentan con una cantidad disponibles de bicicletas mayor a N. El valor debe ser cargado en el `valorInt` del objeto Retorno.

Retornos posibles	
OK	Si indica la cantidad
ERROR	1. Si $n \leq 1$
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

Formato: `cantidad`

Ejemplo: `8`

3.7. Ocupación promedio por barrio

Firma: Retorno `ocupacionPromedioXBarrio();`

Descripción: Para cada **barrio**, porcentaje promedio de **ocupación** = (bicis ancladas / capacidad total del barrio) * 100, con enteros redondeados. Orden alfabético por barrio.

Retornos posibles	
OK	Si listan los libros más prestados
ERROR	
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

Formato: `barrio1#porcentaje|barrio2#porcentaje`

Ejemplo: `Aguada#45|Pocitos#67`

3.8. Ranking por tipo de uso

Firma: Retorno rankingTiposPorUso();

Descripción: Cantidad total de alquileres por **tipo de bici**, **ordenado desc** por cantidad y luego alfabético por tipo.

Retornos posibles	
OK	Si retorna el ranking
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

Formato: tipo#cantidad|...

Ejemplo: URBANA#22|MOUNTAIN#4|ELECTRICA#1

3.9. Usuarios en espera por alquiler

Firma: Retorno usuariosEnEspera(String nombreEstacion);

Descripción: Se deberá mostrar los usuarios que están esperando por alquiler en una determinada estación, en el orden de llegada (primero se muestra quien llegó primero).

Retornos posibles	
OK	Si muestran los usuarios que están esperando por alquiler
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

Formato: ci1|ci2

3.10. Usuario con mayor cantidad de alquileres

Firma: Retorno usuarioMayor();

Descripción: Se indica la ci del usuario que cuenta con la mayor cantidad de alquileres. En caso de haber más de uno, se muestra el de cédula más pequeña.

Retornos posibles	
OK	Si muestra el usuario mayor
ERROR	No hay errores
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

Formato: ci1

PRIMERA ENTREGA

Para la primera entrega se solicita:

- 1) Justificación de las estructuras seleccionadas para resolver TODO el problema planteado. Justificar las decisiones de diseño adoptadas (las mismas deben estar fundamentadas a partir de los requerimientos del problema).
- 2) Implementar las operaciones 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3, 3.4
- 3) Realizar y entregar juego de pruebas que evidencien el correcto funcionamiento de dichas operaciones (las entregadas por el docente y las propias del equipo). Se deberá adjuntar una documentación con la justificación y – en caso de existir – indicar las funcionalidades que no cumplen con los requisitos planteados.

Nota1: previo a la semana de la primera entrega, se compartirá un juego de pruebas reducido.

Nota2: en la lectura de la segunda entrega, se podrían incluir/modificar funcionalidades existentes (en dicho caso, se notificará formalmente)

SEGUNDA ENTREGA

Para la segunda entrega se solicita:

- 1) Justificación de las estructuras seleccionadas - ajustada - final.
- 2) Implementar de TODAS las operaciones de la interfaz “IObligatorio”
- 3) Realizar y entregar juego de pruebas definitivo que evidencie el correcto funcionamiento de TODAS las operaciones.

Contenido de ambas entregas (Se debe subir un único zip o rar con):

- Proyecto con las operaciones implementadas respetando las buenas prácticas y estándares vistos en el curso.
 - La implementación debe respetar la interfaz provista IObligatorio.
 - Incluir en la parte superior de la clase Sistema los nombres y números de estudiante del equipo.
 - Conjunto de pruebas unitarias realizadas por el equipo.
- Documentación: Entregar un documento PDF con una carátula con información de la materia y del equipo. En el cuerpo del documento agregar la descripción de las estructuras utilizadas y una justificación de la adecuación de dichas estructuras de acuerdo con lo solicitado.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

- **Obligatorios**

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo".

Cualquier integrante podrá:

- **Modificar la integración del equipo.**
- **Subir el archivo de la entrega.**

5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails, alamon@ort.edu.uy o rodriguez_mb@ort.edu.uy; o telefónicamente al 29021505 - int 1156 u 1138